



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE

IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- Denominación del Programa de Formación: **ADSO. Análisis y Desarrollo de Software.**
- Código del Programa de Formación: **228118_V1.**
- Nombre del Proyecto: **2386943. Construcción de software para integrar tecnologías orientadas a servicios.**
- Fase del Proyecto: **Ejecución.**
- Actividad de Proyecto: **Codificar los Módulos del Software.**
- Competencia: **220501096. Construcción del software de acuerdo con el diseño y metodologías de desarrollo.**
- Resultados de Aprendizaje Alcanzar: **220501096_04. Codificar el software de acuerdo con el diseño establecido.**
- Duración de la Guía: **600 Horas.**

2. PRESENTACIÓN

En el dinámico mercado laboral de las Industrias 4.0, las organizaciones ya no solo buscan digitalizar sus procesos, sino que demandan soluciones altamente accesibles y conectadas que operen en tiempo real a través de la web y de dispositivos móviles. Tras haber superado las etapas de identificación de necesidades y diseño arquitectónico de su proyecto, este es el momento cumbre donde los requerimientos técnicos y los modelos lógicos se transforman en código fuente funcional, permitiendo que la propuesta de software que ha venido construyendo desde el primer trimestre deje de ser un plan documental para convertirse en un producto tecnológico tangible.

La codificación profesional es el motor que impulsa la competitividad de las empresas tecnológicas a nivel mundial. Escribir código de alta calidad no es una tarea mecánica; requiere un dominio profundo de la lógica y la aplicación rigurosa de saberes técnicos como el Paradigma Orientado a Objetos, el uso de lenguajes de última generación (Java, Python, PHP, C# o Ruby), el manejo de Frameworks, la implementación de



Servicios Web (REST/SOAP) y la integración de protocolos de seguridad para blindar la información contra amenazas críticas como el SQL Injection.

Esta guía es el eje central para alcanzar la competencia de **[220501096] Construcción del software de acuerdo con el diseño y metodologías de desarrollo**. Dicha competencia se integra por cinco resultados de aprendizaje: [01] Planear actividades de construcción, [02] Construir la base de datos, [03] Crear componentes front-end, **[04] Codificar el software de acuerdo con el diseño establecido** y [05] Realizar pruebas al software. El desarrollo de esta guía se enfoca específicamente en el cuarto resultado, donde usted materializará la lógica de negocio y la integración de todos los módulos que componen su solución.

Al finalizar las actividades propuestas, usted estará en capacidad de codificar módulos de software profesionales para entornos web y móviles, creando servicios web para disponer de métodos reutilizables y utilizando herramientas de control de versiones como GIT para asegurar la trazabilidad y colaboración en su código. Lo invitamos a asumir este proceso con rigor técnico, pues de su destreza en la programación depende que el software desarrollado sea escalable, seguro y cumpla con los estándares de calidad que el sector productivo exige hoy en día.

3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

Esta guía propone cuatro momentos de aprendizaje (reflexión inicial, contextualización, apropiación y transferencia) para alcanzar el resultado de aprendizaje “220501096_04 - Codificar el software de acuerdo con el diseño establecido”. Al finalizar la guía, el aprendiz estará en capacidad de:

- [01]** Codificar módulos de software stand-alone, web y móviles aplicando estándares profesionales.
- [02]** Implementar la lógica de negocio siguiendo el paradigma orientado a objetos.
- [03]** Crear y documentar servicios web (APIs) para la integración de componentes.
- [04]** Integrar protocolos de seguridad y encriptación en el código fuente.
- [05]** Gestionar el código fuente mediante sistemas de control de versiones (GIT/GitHub).
- [06]** Incorporar tecnologías emergentes según los propósitos del sistema.

Se recomienda al aprendiz que, para el correcto desarrollo de las actividades, tenga en cuenta lo siguiente: [01] Solicite al Instructor Técnico el Plan Concertado de Trabajo para la entrega de las evidencias



de aprendizaje, de tal forma que le permita gestionar su tiempo y esfuerzo. [02] Lea cuidadosamente la información suministrada en esta guía y en cada una de las actividades de aprendizaje propuestas; las cuales, encontrará en la carpeta “AA - Actividades de Aprendizaje”. [03] Consulte los recursos multimedia (documentos, imágenes, vídeos, etc.) que encontrará en la carpeta “MA - Material de Aprendizaje”. [04] Consulte los referentes bibliográficos de la guía, así como los que se encuentran en los cuestionarios, talleres, casos de estudio, entre otros. [05] Profundice sobre los saberes aquí expuestos, consultando a expertos y/o material externo (biblioteca Sena, libros, revistas, sitios web, etc.). [06] Cuando enfrente dudas, apóyese en los instructores técnicos, de inglés y transversales, así como de sus compañeros de Ficha y GAES.

Para tener éxito en el desarrollo de la guía, el SENA le proporcionará los siguientes recursos:

Ambiente Requerido: Espacio de trabajo para 30 aprendices, con mesas, sillas, tablero y dotado con equipos de cómputo con conexión permanente a internet.

Materiales: Computadores de escritorio y portátiles, software de aplicación para realizar entregas y visualizar material digital (Entornos Integrados de Desarrollo, servidor de aplicaciones y gestores de bases de datos. Además, editores de texto, gráficos y de vídeo).

Resumen Global de la Guía (220501096_04)

Momento de Aprendizaje	Horas de Trabajo Directo	Horas de Trabajo Autónomo	Total Horas
3.1. Reflexión inicial	16 horas	2 horas	18 horas
3.2. Contextualización	60 horas	12 horas	72 horas
3.3. Apropiación	342 horas	48 horas	390 horas
3.4. Transferencia	110 horas	10 horas	120 horas
TOTAL GENERAL	528 horas	72 horas	600 horas

3.1. Actividades de Reflexión inicial

Imagine que trabaja en un Proyecto Software desarrollando una plataforma de comercio electrónico. Debido a la urgencia, se decide omitir los estándares de codificación y no configurar el control de versiones (GIT). Días después, se detecta una vulnerabilidad crítica y, al no existir documentación ni historial en GIT, es imposible saber qué cambios causaron el error o volver a una versión estable del sistema.



Descripción de la Actividad: Estimado Aprendiz, de acuerdo con la situación expuesta anteriormente, solucione el cuestionario denominado “AA01_EC01 - Cuestionario I.docx”, ubicado en la carpeta “AA - Actividades de Aprendizaje / AA01 - Reflexión Inicial”.

Tiempo estimado: 18 Horas.

3.2. Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje

Momento: “Antes de Aprender”

Estimado Aprendiz, como ha notado para codificar una solución software profesional orientada a entornos web y móviles, existe un cuerpo de conocimientos previos que debe comprender antes de iniciar la construcción física del producto; como señala la literatura técnica:

"La ingeniería de software comprende un proceso, un conjunto de métodos y un abanico de herramientas que permiten a los profesionales construir software de cómputo de alta calidad. Este enfoque sistemático y disciplinado es lo que diferencia a un programador profesional de un aficionado, permitiendo que la solución sea mantenible y segura" (Pressman, 2010, p. 4).

Por tal razón, la presente guía le proporciona suficiente información en formato multimedia que, encontrará en la carpeta “MA - Material de Aprendizaje”; sin embargo, se sugiere complementar la información con consultas a expertos y/o material externo (biblioteca Sena, libros, sitios web, libros, revistas, etc.), con el ánimo de encontrar la mejor solución. A continuación, se presentan los fundamentos teóricos que sustentan los ejes temáticos de esta fase:

3.2.1. Entornos de desarrollo

La selección de herramientas adecuadas es el primer paso hacia la calidad. Según Pressman (2010), un entorno de ingeniería de software debe proporcionar un conjunto de herramientas integradas que apoyen todas las actividades del proceso, desde la gestión hasta la codificación técnica. Un IDE moderno no es solo un editor, sino un ecosistema que facilita la depuración y la integración continua del código fuente.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA01 - Entornos de desarrollo”].



3.2.2. Programación Orientada a Objetos

Este paradigma es el pilar de la escalabilidad moderna. Joyanes Aguilar (2008) sostiene que el modelo orientado a objetos facilita la creación de software complejo mediante el uso de clases y objetos que encapsulan datos y comportamiento. La abstracción lógica permite que el desarrollador represente entidades del mundo real en un lenguaje que la computadora pueda procesar con eficiencia.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA02 - Programación Orientada a Objetos”].

3.2.3. Frameworks back_end

El uso de marcos de trabajo acelera la entrega de valor. Como indica la literatura en Ingeniería Web, los frameworks proporcionan una infraestructura reutilizable que resuelve problemas comunes de arquitectura, permitiendo que el equipo se enfoque en la lógica de negocio específica del cliente. Su implementación garantiza que el software siga patrones de diseño probados por la industria.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA03 - Frameworks back_end”].

3.2.4. Seguridad de la aplicación

La seguridad no es un añadido, sino una característica intrínseca. Piattini et al. (2007) enfatizan que el aseguramiento de la calidad incluye la protección de los activos de información contra accesos no autorizados y vulnerabilidades críticas como la inyección de código. Codificar de forma segura implica validar rigurosamente cada entrada de datos en el sistema.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA04 - Seguridad de la aplicación”].



3.2.5. Sistema Control de Versiones

La trazabilidad es vital en el trabajo colaborativo. El estándar SWEBOK (IEEE, 2004) define la gestión de la configuración como la disciplina de identificar la configuración del software en puntos definidos del tiempo para controlar sistemáticamente los cambios. Herramientas como GIT permiten que múltiples desarrolladores trabajen sobre un mismo código sin pérdida de integridad.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA05 - Sistema Control de Versiones”].

3.2.6. Servidores

El entorno de ejecución determina el alcance de la solución. Varón (2021) explica que la infraestructura de TI para ingeniería web requiere servidores configurados para soportar arquitecturas distribuidas, garantizando que los servicios estén disponibles para el consumo desde diversos clientes móviles y web.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA06 - Servidores de aplicaciones”].

3.2.7. Servicios web

La interoperabilidad define al software moderno. Según Sommerville (2005), la arquitectura orientada a servicios permite que los componentes de software se comuniquen de forma independiente a través de protocolos estándar como REST, intercambiando datos en formatos ligeros como JSON.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA07 - Servicios web”].



3.2.8. Cloud Computing

La nube ha transformado el despliegue de aplicaciones. Joyanes Aguilar (2012) define este modelo como una estrategia donde el software se ofrece como servicio, permitiendo que la potencia de cómputo y el almacenamiento sean elásticos y accesibles desde cualquier lugar del mundo.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA08 - Cloud Computing”].

3.2.9. Internet of Things

La integración del mundo físico y digital es ahora una realidad técnica. El concepto de IoT implica la conexión de objetos cotidianos a la red para la recolección masiva de datos, lo cual exige que el software de backend sea capaz de procesar eventos en tiempo real de manera masiva.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA09 - Internet of Things”].

3.2.10. Blockchain

Para procesos que requieren inmutabilidad, el registro distribuido es la solución. Esta tecnología permite asegurar que cada transacción realizada en el software sea auditable y resistente a la manipulación, proporcionando una capa superior de confianza en entornos financieros y logísticos.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA10 - Blockchain”].



3.2.11. Machine Learning

La inteligencia artificial permite que el software aprenda de los datos. La integración de modelos inteligentes en las aplicaciones permite automatizar la toma de decisiones y personalizar la experiencia del usuario final basándose en patrones de comportamiento históricos.

[Para más información, consulte el contenido multimedia ubicado en la carpeta “MA - Material de Aprendizaje / MA11 - Machine Learning”].

Descripción de la Actividad: Estimado Aprendiz, con el fin de diagnosticar el conocimiento adquirido, solucione el cuestionario denominado “AA02_EC02 - Cuestionario II.docx”, el cual está ubicado en “AA - Actividades de Aprendizaje / AA02 - Contextualización”.

Tiempo estimado: 72 Horas.

3.3. Actividades de apropiación del conocimiento (Conceptualización y Teorización)

Momento: “Mientras Aprendo”

En este punto ya ha adquirido los conocimientos previos necesarios para codificar módulos de software (web y móviles), implementar servicios web (APIs), integrar esquemas de seguridad y gestionar el código fuente mediante sistemas de control de versiones en la construcción física del producto; entonces, es hora de ponerlos en evidencia a través de la práctica.

Descripción de la Actividad: Estimado Aprendiz, desarrolle el taller denominado “AA03_ED01 - Taller I.docx”; cada punto del taller, está acompañado por “Prácticas Guiadas”. Tanto el taller como las prácticas, están ubicados en la carpeta “AA - Actividades de Aprendizaje / AA03 - Apropiación”.

Tiempo estimado: 390 Horas.



3.4. Actividades de transferencia de conocimiento

Momento: “Después de Aprender”

Descripción de la Actividad: Estimado aprendiz, con lo aprendido a lo largo de esta guía, ya tiene las bases suficientes para transferir los conocimientos adquiridos a su proyecto formativo: [01] De forma individual, responda la Evaluación denominada “AA04_EC03 – Evaluación I”; [02] En GAES, prepare los entregables del proyecto formativo, tomando en cuenta la lista de chequeo denominada “AA04_EP01 - Entregables de Proyecto.docx”. La evaluación y la lista de chequeo, las podrá encontrar en la carpeta “AA - Actividades de Aprendizaje / AA04 - Transferencia”.

Tiempo estimado: 120 Horas.

4. ACTIVIDADES DE EVALUACIÓN

Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
Evidencias de Conocimiento: - AA01_EC01 - Cuestionario I - AA02_EC02 - Cuestionario II - AA04_EC03 - Evaluación I Evidencias de Desempeño: - AA03_ED01 - Taller I - AA03_ED02 - Práctica Guiada - AA03_ED03 - Práctica Guiada - AA03_ED04 - Práctica Guiada - AA03_ED05 - Práctica Guiada - AA03_ED06 - Práctica Guiada - AA03_ED07 - Práctica Guiada - AA03_ED08 - Práctica Guiada Evidencias de Producto: - AA04_EP01 - Entregables del Proyecto	- Codifica los módulos del software stand-alone, web y móvil, de acuerdo con las especificaciones del diseño y el estándar de codificación. - Crea servicios web para disponer de métodos reutilizables en el software. - Integra los módulos del software de acuerdo con los propósitos del sistema. - Incorpora tecnologías emergentes y disruptivas de acuerdo con los propósitos del software.	Cuestionarios y Evaluación Taller y Práctica Guiada Lista de Chequeo



5. GLOSARIO DE TÉRMINOS

API (Application Programming Interface): Conjunto de definiciones y protocolos que permite que diferentes aplicaciones de software se comuniquen entre sí, facilitando la integración de servicios web bajo estándares como REST o SOAP.

Blockchain: Tecnología de registro distribuido e inmutable que asegura que cada transacción realizada en el software sea auditable y resistente a la manipulación, proporcionando confianza en entornos digitales.

Cloud Computing: Estrategia de computación donde el software, la potencia de cómputo y el almacenamiento se ofrecen como servicios elásticos y accesibles desde cualquier lugar a través de internet.

Codificación de Software: Proceso técnico de transformar requerimientos técnicos y modelos lógicos en código fuente funcional, permitiendo que una propuesta de software se convierta en un producto tecnológico tangible.

Control de Versiones (GIT): Sistema diseñado para gestionar sistemáticamente los cambios realizados en el código fuente, permitiendo la trazabilidad, el trabajo colaborativo en equipo y la recuperación de versiones estables.

Framework: Estructura o marco de trabajo predefinido que proporciona una infraestructura reutilizable de código y patrones de diseño, permitiendo a los desarrolladores agilizar la construcción de la lógica de negocio.

Internet of Things (IoT): Red de objetos físicos conectados a internet que recolectan y procesan datos en tiempo real, integrándose con sistemas de backend para la automatización y monitoreo.

Machine Learning: Disciplina de la inteligencia artificial que permite que el software aprenda de patrones de datos históricos para automatizar la toma de decisiones y mejorar la experiencia del usuario de forma autónoma.



Programación Orientada a Objetos (POO): Paradigma de programación basado en el concepto de "objetos" y "clases", que permite organizar el código mediante la encapsulación de datos y comportamientos para crear software escalable.

SQL Injection: Vulnerabilidad crítica de seguridad que ocurre cuando un atacante logra insertar código SQL malicioso en las entradas de datos, permitiéndole manipular la base de datos de la aplicación.

6. REFERENTES BIBLIOGRÁFICOS

IEEE Computer Society. (2004). SWEBOK: Guide to the Software Engineering Body of Knowledge. IEEE Press.

Joyanes Aguilar, L. (2008). Fundamentos de programación: Algoritmos, estructuras de datos y objetos (4ta ed.). McGraw-Hill.

Joyanes Aguilar, L. (2012). Computación en la nube: Estrategias de Cloud Computing en las empresas. Alfaomega.

Piattini, M. G., García Rubio, F. O., & Caballero, I. (2007). Calidad de sistemas informáticos. Alfaomega.

Pressman, R. S. (2010). Ingeniería del software: Un enfoque práctico (7ma ed.). McGraw-Hill.

Sommerville, I. (2005). Ingeniería de software (7ma ed.). Pearson Educación.

Varón Quintero, J. E. (2021). Técnicas para formular proyectos de software e ingeniería web. Ecoe Ediciones

7. CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	Albeiro Ramos Villamil	Instructor	CEET	23/04/2026

8. CONTROL DE CAMBIOS

	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)	Albeiro Ramos Villamil	Instructor	CEET	23/04/2026	Diseño de la Guía